

# Advanced Packet Sniffer & ARP Spoofing Detector

Project Documentation & Source Code Report

## 1. Project Overview

This project focuses on network security auditing by implementing a real-time Packet Sniffer and a behavioral ARP Spoofing Detector. It targets security vulnerabilities in local area networks (LANs), specifically tackling Man-in-the-Middle (MitM) positioning attacks caused by Address Resolution Protocol (ARP) poisoning.

|                     |                                                 |
|---------------------|-------------------------------------------------|
| Project Title       | Advanced Packet Sniffer & ARP Spoofing Detector |
| Developer Name      | Makesh Kumar                                    |
| Operating System    | Linux Mint 22 (Ubuntu Noble Base)               |
| Development Library | Python 3.12 & Scapy Engine                      |
| Tested Interface    | wlp2s0 (Wireless Network Card / Mobile Hotspot) |

## 2. System Architecture & Component Structure

The project is built modularly using Python 3.12 within an isolated Linux system virtual environment (venv) to comply with modern Python standards (PEP 668).

```
arp_project/
├── venv/                # Isolated Python environment
├── main.py              # Command-Line Interface (CLI) application router
├── sniffer.py           # Layer 3 / Layer 4 Packet Capturing Engine
├── detector.py          # Layer 2 Stateful ARP Monitoring Engine
├── logs/
│   └── attack_alerts.log # Persistent file logging cryptographic/MAC anomalies
```

## 3. Core Functionalities

- Live Packet Sniffing:** Intercepts real-time network packets directly from the active wireless network interface, resolving IP boundaries and parsing Layer 4 protocols (TCP, UDP).
- Stateful ARP Monitoring:** Maintains a dynamic runtime Look-Up Table (LUT) mapping IP addresses to their unique hardware MAC addresses.

- **Anomalous Behavior Flagging:** Generates immediate visually distinct terminal warnings (Red ANSI Alerts) if a single IP address broadcasts conflicting physical MAC addresses.
- **Persistent Threat Logging:** Automatically appends security incident metrics with precise timestamps into an external log file.

## 4. Source Code Implementation

### A. Interface Router (main.py)

```
import sys
import sniffer
import detector

def main():
    print("="*50)
    print("  ADVANCED PACKET SNIFFER & ARP SPOOFING DETECTOR  ")
    print("="*50)
    print("1. Run Live Packet Sniffer\n2. Run ARP Spoofing Detector\n3. Exit")
    choice = input("Select an option (1-3): ").strip()

    if choice in ['1', '2']:
        interface = input("Enter your network interface name (e.g., wlp2s0): ").strip()
        if choice == '1':
            sniffer.start_sniffing(interface)
        else:
            detector.start_detector(interface)
    else:
        sys.exit(0)

if __name__ == "__main__":
    main()
```

### B. Packet Sniffer Module (sniffer.py)

```
import scapy.all as scapy

def process_packet(packet):
    if packet.haslayer(scapy.IP):
        src_ip = packet[scapy.IP].src
        dst_ip = packet[scapy.IP].dst
        proto = "TCP" if packet[scapy.IP].proto == 6 else "UDP" if packet[scapy.IP].proto == 17 else "Other"
        print(f"[+] Packet: {src_ip} -> {dst_ip} | {proto}")

def start_sniffing(interface):
    print(f"[*] Starting Packet Sniffer on {interface}...")
    scapy.sniff(iface=interface, store=False, prn=process_packet)
```

## C. ARP Detector Engine (detector.py)

```
import scapy.all as scapy
from datetime import datetime

ip_mac_table = {}
LOG_FILE = "logs/attack_alerts.log"

def log_incident(message):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(LOG_FILE, "a") as f:
        f.write(f"[{timestamp}] {message}\n")

def process_arp_packet(packet):
    if packet.haslayer(scapy.ARP) and packet[scapy.ARP].op == 2:
        src_ip = packet[scapy.ARP].psrc
        src_mac = packet[scapy.ARP].hwsrc

        if src_ip not in ip_mac_table:
            ip_mac_table[src_ip] = src_mac
        elif ip_mac_table[src_ip] != src_mac:
            old_mac = ip_mac_table[src_ip]
            alert_msg = f"⚠️ ALERT: ARP Spoofing Detected! IP [{src_ip}] changed from [{old_mac}] to [{src_mac}]!"
            print(f"\033[91m{alert_msg}\033[0m")
            log_incident(alert_msg)

def start_detector(interface):
    print(f"[*] Starting ARP Spoof Detector on {interface}...")
    scapy.sniff(iface=interface, store=False, prn=process_arp_packet, filter="arp")
```

## 5. Empirical Results & Verification

During live network simulation on the **wlp2s0** interface, the stateful engine successfully identified hardware address manipulation. The system immediately raised a visual Red Alert on the console and correctly committed the threat metrics into the persistent log file (`logs/attack_alerts.log`), verifying total defense capability.